

Article

Recommender Systems for Multimodal Transportation in Smart Sustainable Cities

Houda El Bouhissi ^{1,*} , Thomas Hanne ^{2,*}  and Mounia Madadi ³ ¹ LIMED Laboratory, Faculty of Exact Sciences, University of Bejaia, Bejaia 06000, Algeria² Institute for Information Systems, University of Applied Sciences and Arts Northwestern Switzerland, Olten 4600, Switzerland³ École Supérieure en Sciences et Technologies de l'Informatique et du Numérique, RN 75, Bejaia 06300, Algeria

* Correspondence: houda.elbouhissi@gmail.com (H.E.B.); thomas.hanne@fhnw.ch (T.H.)

Abstract

Transportation recommendation systems (RS)s have garnered significant attention owing to their ongoing potential for enhancement. One of the key innovations in this domain is multimodal transportation RSs, which suggest travel routes using a combination of different transportation modes. In this paper, a multimodal transportation RS is introduced, which recommends optimized trajectories based on user preferences. The system involves two main steps, trajectory generation and ranking. In the first step, Particle Swarm Optimization (PSO) is used to find optimal trajectory combinations between the origin and destination, followed by post-processing. In the second step, the generated trajectory is evaluated using a RankNet model trained on historical user data with a content-based approach. The results demonstrate the system's ability to generate feasible trajectories and provide precise recommendations. The results enable an efficient usage and convenient user experiences and may foster the broader use of public transportation combined with other transport modes addressing the objectives of smart and sustainable future cities.

Keywords: recommendation system; multimodal transport; Particle Swarm Optimization; sustainable smart cities



Academic Editors: Domenico Gattuso, Giuseppe Guido and Giovanni Leonardi

Received: 28 August 2025

Revised: 13 November 2025

Accepted: 20 November 2025

Published: 2 December 2025

Citation: El Bouhissi, H.; Hanne, T.; Madadi, M. Recommender Systems for Multimodal Transportation in Smart Sustainable Cities. *Sustainability* **2025**, *17*, 10810. <https://doi.org/10.3390/su172310810>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The plethora of public transportation alternatives in urban settings, comprising metro, tramway, buses, trains, and taxis, provides a multitude of travel alternatives. Navigating through these alternatives can prove to be intricate, particularly for individuals who are unfamiliar with the city, whether they are tourists or residents from other metropolitan areas. Finding the optimal public transportation route from one location to another, considering factors such as cost, comfort, and duration of travel, is a challenging task. Thus, the support of travellers and the need for recommendation systems (RS)s become apparent [1–3], particularly within the context of smart cities.

The transportation sector greatly benefits from the rapid evolution of these systems, which demonstrate an increasing ability to provide personalized recommendations to each user. A concrete example of this utility lies in the proposal of the best trajectory, considering the users' past travel choices, budget preferences, and time and comfort preferences. In connected urban environments or smart cities [4–6], this convergence between the combination of available transportation modes and intelligent recommendations offers a particularly relevant solution to simplify and optimize city travel.

Multimodal transportation systems are key indicators of the advancement of smart transportation [7]. These systems are defined in various ways, such as the shipment of cargo and the movement of people involving more than one mode of transport during a single seamless journey. Another definition is the combination of modes of transport, usually ship, truck, or rail, to transport freight. All these definitions refer to systems that use a combination of different transportation methods to travel from a starting point to a destination.

Moving towards a multimodal transportation approach requires addressing various issues and adapting organizational structures to support various transportation modes. Conventional planning often focuses on automobile travel. This approach uses models designed for road transport, emphasizing speed, costs, or congestion reduction. Multimodal planning, on the other hand, integrates several transportation modes, encouraging higher land-use density and mixed-use development. It prioritizes modal diversity, necessitating more intricate analysis tools to evaluate the quality and inter-connectivity of various transportation modes.

Statewide multimodal transportation planning faces several significant obstacles. First, there is a growing need for performance-based planning, where transportation agencies are responsible for measurable results, not just output. This necessitates the development of neutral performance measures utilizing readily available data. Second, greater public and stakeholder participation is essential to ensure that diverse interests, including minority and low-income populations, are fairly represented. Another significant obstacle lies in the integration of advanced technology, which has the potential to transform data processing and analysis; however, it necessitates enhanced tools and methodologies. Furthermore, environmental sustainability and the fair distribution of transportation benefits are becoming increasingly important. In addition, it is essential to coordinate planning efforts across the local, regional, and multi-state levels to avoid political impasse and guarantee comprehensive solutions. Finally, it remains a significant hurdle to recruit, train, and retain skilled professionals with the necessary technical expertise.

In the context of smart cities, multimodal transportation has been identified as a pivotal element, facilitating the integration of diverse transportation modes, including buses, trains, bicycles, and ride-sharing services, into a unified and efficient mobility infrastructure [8–11]. This integration is imperative for addressing urban challenges, including traffic congestion, pollution, and the inefficient coverage of public transport. The provision of flexible, accessible, and eco-friendly travel options by multimodal systems has been demonstrated to enhance the overall user experience, promote sustainable mobility, and facilitate real-time decision-making through the integration of smart technologies. In the context of smart cities, multimodal transport has been shown to improve connectivity and reduce travel time. Furthermore, it aligns with the broader goals of environmental sustainability and digital urban governance.

Smart cities use technologies including the Internet of Things, artificial intelligence, and data analytics. The implementation of innovative transportation models can significantly benefit human beings and provide valuable resources to citizen convenience [11].

The need to offer smarter mobility solutions, along with the growing complexity of urban transport systems, means that recommending optimized multimodal journeys has become a key issue. This problem involves taking several, often conflicting, criteria into account simultaneously, such as travel time, cost, accessibility, and environmental impact. To address this issue effectively, powerful optimization methods are needed that can process large volumes of heterogeneous data and adapt to dynamic constraints. With this in mind, our approach is based on the PSO (Particle Swarm Optimization) algorithm, which is recognized for its fast convergence and ability to efficiently explore large search spaces. A

comparative study with other bio-inspired algorithms, such as Gray Wolf Optimization (GWO), Lion Optimization Algorithm (LOA), and Genetic algorithm (GE), demonstrated the superiority of PSO in terms of recommendation quality, result stability, and computation time, leading to the selection of PSO. Therefore, integrating PSO into our recommendation system is a relevant method for improving the model's overall performance.

This paper concerns the transportation domain in a smart city and addresses the issue of recommending trajectories by integrating various modes of transportation within a smart city and presenting them to users based on their personal preferences regarding cost, time, and preferred mode of transport. Initially, the trajectories were constructed by employing PSO, once the trajectories are refined to ensure presentability, they are ranked for the user using a RankNet model trained on the user's previous journeys.

The rest of the paper is organized as follows. Section 2 discusses the background and provides a literature review, which highlights trends and key contributions in multimodal transportation recommendation systems. Section 3 presents our proposed approach for integrating PSO for trajectory generation and RankNet for ranking. Section 4 discusses the systems implementation and evaluation, including the datasets and tools used. Finally, Section 5 concludes the article with research findings and future insights.

2. Related Works

In its most common definition, an RS is an algorithm that rates items, aiming to identify those with the highest ratings, as they are likely the ones preferred by the user [12]. Subsequently, the system recommends these highly rated items to the user. In addition, it is important to mention that the method of rating varies among different RS. Each system employs its own unique approach to analyzing data sources to establish connections between users and items.

As the methods of generating recommendations vary from one system to another, RSs are categorized into three main classes as shown in Figure 1 that describe the general approaches: (1) Collaborative filtering, (2) content-based filtering, and (3) hybrid filtering, which combines elements of both [13].

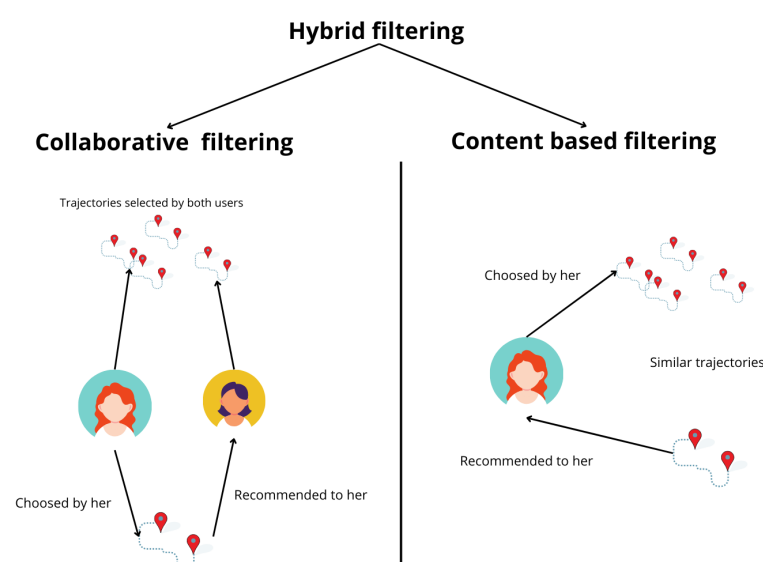


Figure 1. Recommender system types.

2.1. Content-Based Systems

Content-based systems (CB) are based on the intuition that if a user liked a particular item, they will probably like similar ones as well [14]. It works by identifying similarities between items and then recommending those with features similar to those the user has

liked before. To determine the items the user liked in the past, various methods are employed. These can be explicit, such as directly asking the user, or implicit, which involves analyzing user behavior. In this approach, dealing with cold starts is usually not an issue, as every item/user has its attributes/preferences defined once it is entered in the database [15].

2.2. Collaborative Filtering Systems

In collaborative filtering (CF) systems, items are suggested to users based on ratings from others whose behaviors align closely with theirs [14]. This method stands out as the most popular and widely utilized one, with significant advancements in both academia and industry. While this approach is widely used, it has both advantages and disadvantages. One advantage is that it does not require additional information about items or users when adding them to the database, as it purely relies on the interaction between users and items. However, if there is not enough interaction, the system may generate fewer recommendations. CF, in turn, is divided into two general classes: (1) memory-based, which uses the database each time for recommendations, and (2) model-based, which constructs a model for predictions. Similarly to the previous approach, the ratings can be either explicit or implicit [16].

2.3. Hybrid Systems

Both approaches mentioned above have their own set of advantages and disadvantages. To address the limitations of each while capitalizing on their strengths, hybrid systems were developed [14]. These RSs combine elements of both CF and CB into a single structure. There are various methods available for constructing hybrid collaborative systems, which vary according to the data employed.

The review of related studies shows that these works extensively use advanced machine learning (ML) and deep learning (DL) techniques to enhance RSs. For instance, Convolutional Neural Networks (CNN) and gradient-boosted decision trees (GBDT) are commonly employed to handle complex data and improve model performance. Additionally, many studies incorporate graph-based methodologies to model the relationships between users, origin–destination pairs, and transport modes. This approach is exemplified by the use of bipartite graphs [17] and heterogeneous graphs [18].

However, each study adopts a different approach in its recommendation and trajectory generation methods. For recommendation generation, the following approaches were identified:

- Bayesian Learning and Transfer Learning: Ref. [19] utilized a continuous update of user profiles and applied pre-existing knowledge for trajectory recommendations.
- Optimization Modeling: Ref. [20] used a dynamic programming approach for optimizing container multimodal transport systems.
- Joint Representation Learning: Ref. [21] employed a multi-modal transportation graph to learn representations for recommendation.
- Multi-Task Deep Learning Systems: Ref. [22] used a combination of lightweight GBDT and multi-task wide and deep learning (MTWDL) for context-aware recommendations.

For route generation, the following methods were identified:

- Graph Embedding Techniques: Ref. [17] constructed bipartite graphs and transformed nodes into feature vectors.
- Heterogeneous Graph Attention Networks: Ref. [18] utilized a hierarchical attention mechanism to generate node embeddings.
- Time-Dependent Multi-View Transportation Graphs: Ref. [23] transformed transportation networks into multi-view graphs and employed a graph-based contextual encoder.

- **Ripple-Spreading Algorithm:** Ref. [24] utilized an improved ripple-spreading algorithm to find the K shortest paths in intercity multimodal transport networks.

The datasets used also vary significantly from one study to another, with some studies focusing on city-specific data while others utilize broader datasets covering multiple cities and transport modes. Additionally, many datasets are confidential.

The outputs of these systems also differ, aligning with the objectives of the respective applications:

- **Personalization and Context Awareness:** Achieved through dynamic user profiles and consideration of situational factors like weather [19,22].
- **Optimal Transport Strategies:** Effective strategies for container transport systems and other multimodal transport scenarios [20].
- **Unified transport network representation:** Leveraging spatiotemporal dependencies for robust recommendations [23].
- **Efficiency in Computation:** Improved algorithms for faster and optimal path planning [24].

Recently, several studies have focused on transportation in smart cities, with a view to improving the efficiency of urban mobility. Ghalmane and Daoud [25] present a route recommendation framework that uses graph-based deep learning to integrate multiple transportation modes (e.g., bus, tram, bike). The transportation network is modeled as a graph where nodes represent transit stops and edges represent route segments. The authors apply Graph Convolutional Networks (GCNs) to capture spatial dependencies and GRUs to model temporal dynamics. A multi-task learning approach is used to predict various route attributes. These include CO₂ emissions, safety, and distance. These predictions are combined in a weighted scoring function to recommend routes based on user and system priorities. The model also incorporates contextual features such as weather and comfort levels. A case study of the transport network in Strasbourg demonstrates that this method improves the quality of recommendations by balancing efficiency, safety and environmental impact. The approach under discussion here shows how combining graph learning and sequence modeling could result in intelligent, user-aware route planning in smart cities.

Another recommendation framework for multimodal travel plans that explicitly accounts for temporal characteristics and candidate route sets is proposed by Yu et al. [26]. Temporal characteristics refer to time of day, travel-time variations and departure time preferences, while candidate route sets refer to the set of possible multimodal options given the origin, destination and modes. The model learns from historical travel data to understand how satisfactoriness or choice changes depending on temporal traits, such as rush versus non-rush hours, and uses this information to adjust routing recommendations. It also examines how selecting different sets of candidate routes as inputs affects the quality of the recommendation; more diverse and relevant sets lead to better suggestions. The framework combines these influences to re-score or re-rank multimodal journey plans based not simply on static criteria such as shortest time or fewest transfers. In experiments using real urban mobility data, this method was shown to improve recommendation accuracy and relevance compared with baselines that ignore temporal variation or artificially constrain candidate options.

An interesting approach proposed by Jing et al. [27] involves a personalized trajectory recommendation system that integrates real-time and historical driver, vehicle, and road condition preferences. The system uses a hybrid edge-cloud architecture, where the cloud processes historical data to learn long-term patterns, while the edge handles real-time, localized decisions based on the current driver state, vehicle condition, and road emergencies. It calculates section-level scores based on metrics such as usage frequency, fluency, driver

familiarity, and unexpected factors and then ranks entire trajectories. Using real-world taxi data from Shanghai, the system achieved high accuracy (over 95% in many cases) and outperformed a baseline method (ASER) in terms of both recommendation quality and computational efficiency. The model adapts dynamically as new data becomes available, making it suitable for large-scale, real-time trajectory planning. While not explicitly multimodal, the method offers strong potential for trajectory optimization systems that value personalization and responsiveness.

In addition, Gökhan and İlhan [28] suggest a hybrid metaheuristic route planning algorithm that brings together PSO and GWO. This is done to take advantage of both PSO's efficient global search and GWO's exploration/local minima avoidance. The authors test the algorithm in simulated environments using sensor input and compare its performance to using PSO alone and GWO alone. The metrics for comparison include execution time to find the optimal route, route length, number of iterations, and ability to avoid becoming stuck in local minima. The results show that the proposed method achieves significantly lower computation time and iteration count than GWO, while also mitigating PSO's tendency to settle into sub-optimal (local) solutions. Shorter routes are produced, and the algorithm more reliably escapes local minima than PSO alone. They emphasize the benefits of this approach in contexts such as disaster management, where unmanned vehicles must plan routes quickly and reliably in obstructed or unknown environments.

Although all studies aim to improve multimodal transport RSs, they differ significantly in their methodology, data utilization, and specific outcomes. The common thread across these studies is the use of advanced ML techniques and the integration of diverse data sources to provide accurate, personalized, and context-aware recommendations. These similarities and differences highlight the evolving landscape of multimodal transport RSs and the ongoing efforts to improve their performance and user satisfaction.

3. Proposed Approach

In this section, our proposal is presented in detail. The key terms and concepts essential for the study are introduced first. These foundational elements provide the necessary theoretical framework to explore the core aspects addressed here.

- Stations or stops : Stations are the constituent elements of our lines, referring to all public transport stations within the transport network.
- Lines: Lines refer to the different routes of the transport network, regardless of the mode of transport, composed of a set of stations arranged in a specific order.
- Trajectories or Routes: Trajectories represent the sequence of lines, each encompassing all the stops through which one passes, thereby forming a continuous link from the starting point to the destination.

The proposed approach integrates the bioinspired algorithm PSO for trajectory generation and RankNet for trajectory recommendation. The proposal addresses the challenge of efficiently guiding users to their destinations, considering factors such as travel time, cost, and user preferences regarding transportation modes. A detailed breakdown of each step of the approach is provided, including capturing origin–destination pairs, generating optimal paths using PSO, calculating trajectory attributes for recommendation, and utilizing RankNet for trajectory recommendation. Through this approach, our goal is to enhance the effectiveness and accuracy of trajectory RSs, thereby improving the overall user experience in navigating transportation networks.

Recognizing that in certain cities, the selection of the optimal mode of transportation from one location to another is no longer straightforward due to the multitude of available alternatives and numerous potential trajectories, travelers may encounter uncertainty when deciding on their itinerary. Hence, our system provides users with the optimal route for

utilizing public transportation, irrespective of whether it involves a singular mode or a combination, based on their individual preferences regarding cost, time, and preferred travel. The proposed system involves the essential steps as illustrated in Figure 2 to provide this recommendation.

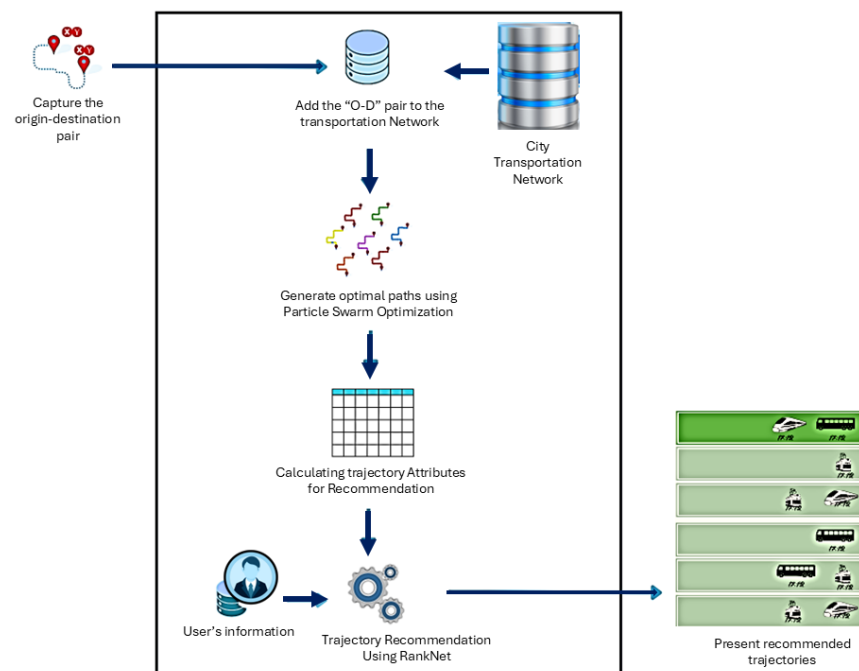


Figure 2. Proposed multimodal transportation approach.

- Capture the user's starting and destination locations in the format (longitude, latitude), then add these two points as the starting and ending nodes, respectively, to our transportation network.
- Generate some optimal paths in terms of the distance between the starting node and the destination node using PSO.
- Once the optimal trajectories have been selected, calculate their necessary attributes for the recommendation, including the percentage of each mode of transportation, the total time, and the total cost.
- Proceed to recommend the best trajectory using RankNet. Subsequently, present the user with the top recommendations, indicating the recommended one and providing details of the trajectories.

Each step is now outlined to clarify its role and significance in the process:

3.1. Capture the Origin–Destination Pair

As soon as the request begins, the user indicates on the city map their starting and destination points. From their input, the coordinates of these two points, namely their longitude and latitude, are retrieved.

Next, links are established between each of these two points and the nearest stations. The objective is to select nearby stations in the network while ensuring that at least one station is chosen. To achieve this, the distances of all stations from the departure and destination locations are calculated, these distances are then sorted, and the first stations are selected. The selection process stops when the distance becomes significantly larger than the average distance of the stations already chosen. A walking link is created between stations if they are sufficiently close. These two newly selected nodes are then designated as the starting and destination nodes. A gridding procedure, as illustrated in Figure 3, is applied

to retain only the stations present within the box defined by the upper and lower bounds of the longitudes and latitudes of the origin–destination pair. This step is essential to reduce the search space. The detailed steps are explained in Algorithm 1, and Figure 4 illustrates the generation of these links, where it is observed that the algorithm will select the stations that are close to the departure and arrival locations, regardless of the type of station.

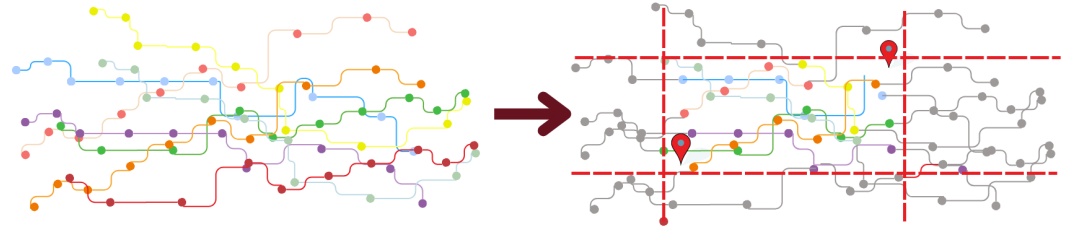


Figure 3. Apply grids to form a box around the origin and destination. Each sequence of nodes (stations) with connections in one color corresponds to a transport line.

Algorithm 1: Neighbor stations selection

Input: Origin–Destination (O-D) coordinates, Transportation network

Output: Two lists of selected stations

Begin

Calculate the distance of each station from the departure station O and the destination station D ; Sort the stations by the distance from the departure station;

Initialize an empty list of selected stations (*selected_stations_O*);

Initialize a variable *Avr* as the maximum distance initially that will contain the average distance of the added stations;

while the distance of the next station is not significantly larger than *Avr* **do**

 Add the new stations to *selected_stations_O*;

 Update the value of *Avr* as the new average of the distances of the selected stations;

end

Sort the stations by the distance from the destination station;

Initialize an empty list of selected stations (*selected_stations_D*);

Initialize *Avr* as the maximum distance initially;

while the distance of the next station is not significantly larger than *Avr* **do**

 Add the new stations to *selected_stations_D*;

 Update the value of *Avr* as the new average of the distances of the selected stations;

end

Return the two lists: *selected_stations_O* and *selected_stations_D*;

End.

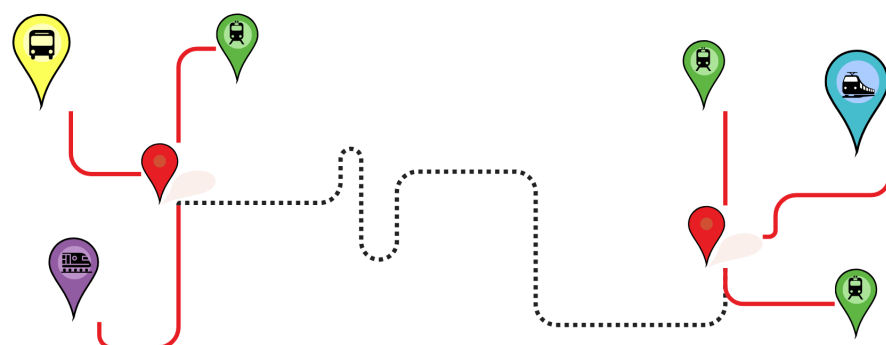


Figure 4. Link creation between the origin–destination pair and the closest stations to them.

3.2. Generate Possible Trajectories Using PSO

To calculate all possible trajectories, our network is stored as a graph $G = (U, V)$, where each station is represented by a node U_i (i number of nodes). These nodes capture the station's location and line ID. Connections between stations are represented by edges E_{ij} . These edges signify the adjacency of stations on the same line and are characterized by the cost of traveling between stations C_{ij} , the time spent to travel between them T_{ij} , and the identifier of the line that connects them. This graph structure, as illustrated in Figure 5, enables the efficient exploration of various trajectories within the transportation network, where a line is represented by a color, so that an arc link between each successive station on the same line in the network can be clearly observed. Finally, the representation of the trajectories that will be generated is shown as a series of arcs between the departure point and the arrival point. The generated trajectories is a list of nodes that need to be visited in that order to reach the destination: $Tr_j = [U_1, U_2, \dots, U_n]$.

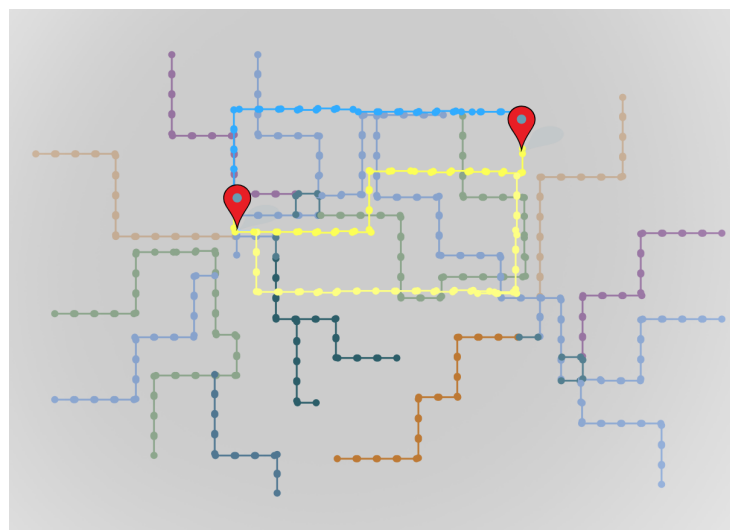


Figure 5. The transportation network representation. Each sequence of nodes (stations) with connections in one color corresponds to a transport line.

The next step involves generating possible trajectories. For this purpose, Particle Swarm Optimization (PSO), a bio-inspired algorithm [29]. PSO is a metaheuristic optimization algorithm that is particularly effective at solving complex problems where the search space is vast, nonlinear, or ill-defined. Because of its ability to quickly converge on optimal or near-optimal solutions, it is widely used in a variety of fields such as engineering, artificial intelligence, energy, logistics, and finance. PSO has the advantages of being easy to implement, inexpensive to compute, and flexible enough to adapt to many constraints and objective functions. The PSO algorithm is applied to obtain the final result of the optimal trajectory. Based on the optimal partition results, representing a city's transportation network as a graph—where stations are nodes and links between stations are edges—causes the search for possible trajectories to grow exponentially. To optimize this search, the PSO algorithm is employed. PSO is an optimization algorithm inspired by the social behavior of birds flocking or fish schooling. In PSO, a group of particles (potential solutions) moves around the search space to find the optimal solution. Each particle adjusts its position based on its own experience and the experience of neighboring particles. The movement of each particle is influenced by its best-known position and the best-known positions of the swarm, as shown in Figure 6, with positions being adjusted using velocity equations. This allows the particles to converge toward the optimal solution over time. PSO was chosen

because swarm intelligence is widely used for its simplicity and effectiveness in solving complex optimization problems.

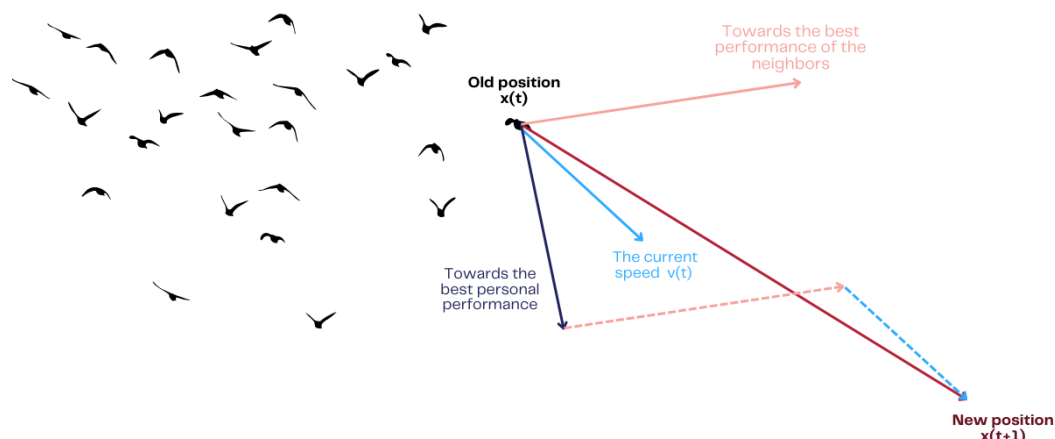


Figure 6. Particle Swarm Optimization principle.

Update Velocity: defines how each particle adjusts its movement based on its past and that of its neighbors. It combines three components—inertia (previous velocity), cognitive (personal best), and social (global best)—to balance exploration and exploitation during the search for the optimal solution (see Formula (1)).

$$v_i^{t+1} = w \cdot v_i^t + c_1 \cdot r_1 \cdot (pbest_i^t - x_i^t) + c_2 \cdot r_2 \cdot (gbest^t - x_i^t) \quad (1)$$

where

- v_i^{t+1} : The new velocity of particle i at iteration $t + 1$.
- w : The inertia weight.
- v_i^t : The current velocity of particle i at iteration t .
- c_1 and c_2 : The learning coefficients.
- r_1 and r_2 : Random numbers between 0 and 1.
- $pbest_i^t$: The best position attained by particle i until iteration t .
- x_i^t : The current position of particle i at iteration t .
- $gbest^t$: The best position attained by the swarm until iteration t .

Update Position: the new location of each particle is determined by adding its updated velocity to its current position. This allows particles to move through the search space toward optimal solutions based on their velocity adjustments from the previous step (see Formula (2)).

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (2)$$

x_i^{t+1} is the new position that should take the particle i at iteration $t + 1$. However, instead of maintaining these new positions, the nearest stations to these positions were selected as the new positions, since it is not possible to position them directly in space. Specifically, the nearest station is not simply chosen. As illustrated in Figure 7, the station closest in the direction of the velocity vector is selected instead. This ensures that the trajectory aligns with the direction indicated by the velocity vector, optimizing the movement toward the destination. In this case, the optimization function represents the distance between the current point and the destination.

For initialization, the particles were positioned at stations close to the origin rather than being initialized randomly in space. At each iteration, the new positions of the particles were calculated using the velocity that had also been calculated using the PSO equations. Starting from the initialization, at each iteration, each trajectory is augmented with the new

station taken by its particle, provided it does not move away from the destination. Finally, the eight trajectories whose last station is closest to the destination are returned, excluding any duplicates. All the steps involved in generating trajectories using the adapted PSO are detailed in Algorithm 2.

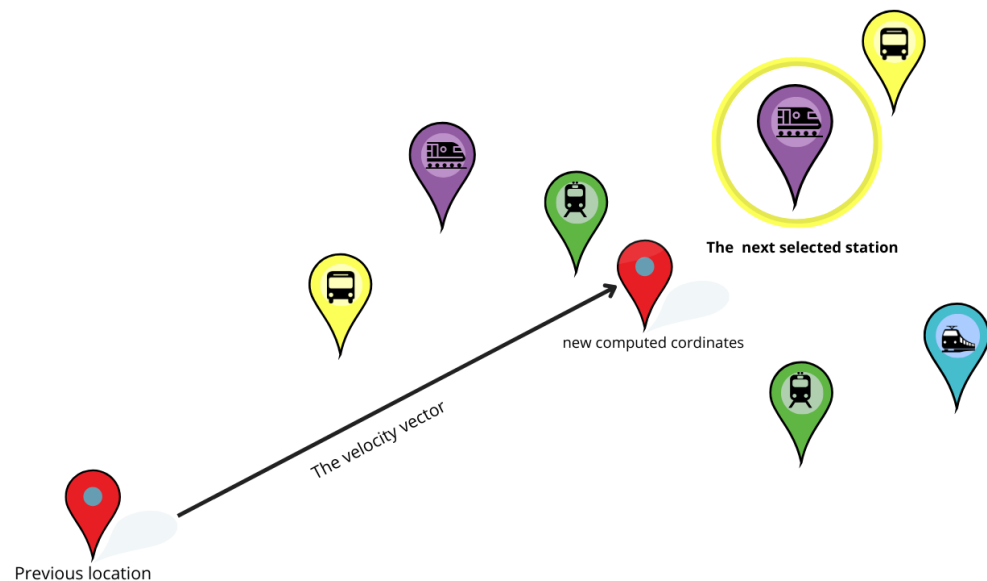


Figure 7. Next station selection.

Algorithm 2: Generating initial trajectories using PSO

Input: Origin and destination identifier, List of the neighbors of the origin and the destination, SwarmSize, MaxIterations, w , c_1 , c_2 , The transportation network

Output: List of best trajectories found

Begin

Initialize a swarm of particles at the origin neighbors;

For each particle, compute its distance from the destination and set each one as the *pbest* of the particles;

Sort the distances and store the smallest 8 as the most optimal ones and set the smallest one as the *gbest*;

while *iter* < *MaxIterations* **or** *at least 3 of the best ones are less than 1 km from the destination* **do**

for each *particle in the swarm* **do**

 Update the velocity of the particle using the PSO equations;

 Update the position of the particle based on the new velocity;

 Update the particle's best-known position (*pbest*) if the fitness improves;

 Update the list of the 8 best particles if the new *pbest* is better than the 8th one in the list;

 Update *gbest* if the particle's fitness is better than the current *gbest*;

if *the particle has reached the destination* **then**

 Remove it from the swarm and do not work on it in next iterations;

end

end

end

Return the list of best trajectories found so far as the solution to the trajectories finding problem;

End.

Since the result is a list of eight trajectories, each represented by a sequence of stations from the starting point to the destination, it is not certain that there is a link between each pair of successive stations in these trajectories. Additional treatments shown in Figure 8 had to be added to these stations. These treatments are as follows:

- First, the destination was added as the final station in the trajectories that had not reached the destination yet.
- Then, combinations of pairs belonging to the same line were searched for. If they existed, then the intermediate stations were removed.
- Next, links between each pair of successive stations were searched for as follows:
 - First, it is checked if they belonged to the same line; if so, their link was marked as 'same line.'
 - If they did not belong to the same line, then it was checked if there was a station that belonged to both a line passing through the first station and another passing through the second station. If such stations existed, then their link was marked as 'line intersection'.
 - If no link from the two previous types was found, then the link between them was marked as 'walk.'
- Then, the intermediate stations were added in both 'intersection of station' and 'same line' types. Duplicated stations were also removed, and if there was any cycle in the trajectory, it was also eliminated. This was done to finally obtain the complete trajectories and a dictionary describing all the lines, each with its stations, to go from the origin to the destination.

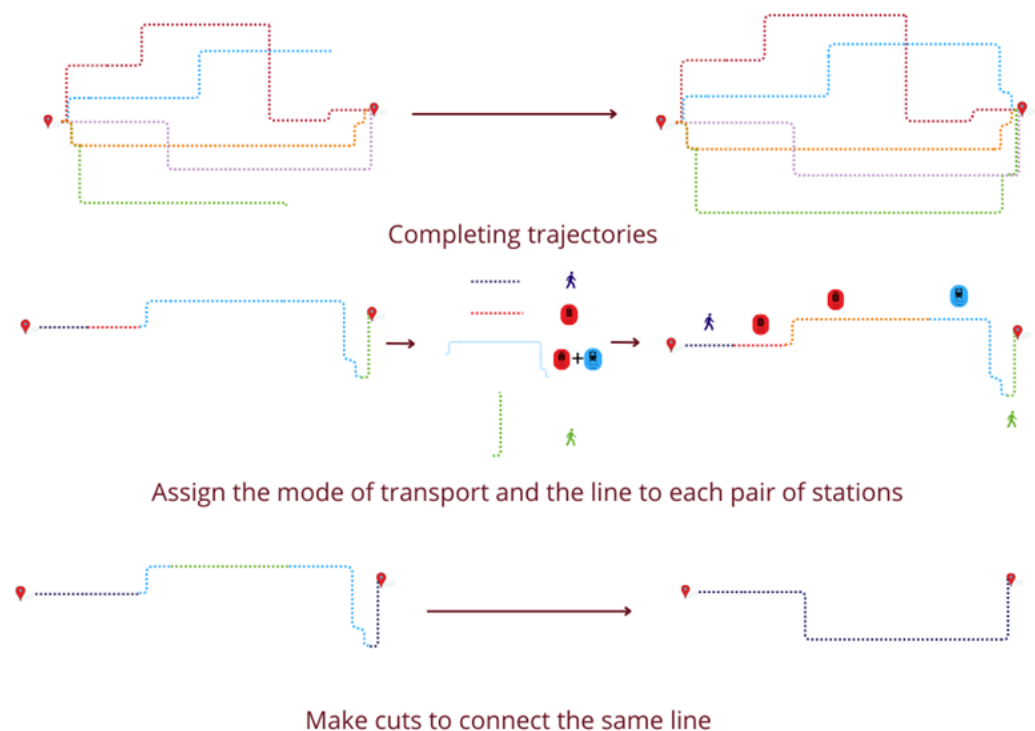


Figure 8. Post-processing steps.

All these post-processing steps are summarized in Algorithm 3.

Algorithm 3: Finalizing the trajectories

Input: Origin and destination identifier, List of the 8 best trajectories, The transportation network

Output: List of best trajectories found

Begin

Remove from the 8 trajectories the duplicated ones;

for each trajectory of the remaining ones **do**

repeat

for each pair of stations in the trajectory **do**

if they belong to the same line **then**

 Remove all intermediate stations between them;

end

end

until no more modifications are made;

for each successive pair of nodes **do**

if they belong to the same line **then**

 Mark their link as ‘same line’;

end

else

 Search for a station that belongs to both a line passing through the first station and another passing through the second station;

if such stations exist **then**

 Mark their link as ‘line intersection’;

end

else

 Mark the link between them as ‘walk’;

end

end

end

Combine the successive nodes that have the same line;

Add the intermediate stations between two stations connected by a ‘same line’ or ‘line intersection’ link;

Organize the trajectory in a dictionary of lines of the trajectory, each one with the type of transportation, the line id and the list of stations;

end

Return the final trajectories;

End.

The PSO algorithm’s ability to dynamically explore the solution space via a group of interacting particles makes it generally robust despite sudden changes in transportation networks, such as route closures or schedule adjustments. This collective nature allows it to adapt quickly to disruptions by redirecting particles to new promising areas of search, promoting convergence toward viable solutions even when network conditions change abruptly. However, the speed and efficiency of this adaptation depend heavily on the frequency of data updates and the PSO parameters, including the inertia velocity and learning coefficients. The accuracy and reliability of the PSO algorithm in generating optimal trajectories for multimodal transportation systems were validated through a combination of simulation experiments and comparative analysis. In particular, the PSO-generated trajectories were put through their paces against a range of benchmark datasets, which represented realistic multimodal transport scenarios. The performance of each was evaluated based

on metrics such as travel time, cost, and route efficiency. The PSO algorithm's solutions were then compared with those obtained using traditional optimization methods and other metaheuristic approaches. This demonstrated that the PSO algorithm produced superior or comparable results in terms of convergence speed and solution quality. Furthermore, sensitivity analyses were conducted to evaluate the robustness of the algorithm in response to varying network conditions and input parameters, ensuring consistent performance across diverse scenarios. These validations collectively confirmed that PSO is accurate and reliable for optimizing complex multimodal transport trajectories.

It should be noted that several optimization algorithms were tested as part of the recommendation of multimodal transport solutions, which involves proposing optimized itineraries by combining different modes of transport (e.g., bus, train, bicycle, walking) while taking into account multiple criteria (e.g., time, cost, comfort, CO₂ emissions). The performance of three algorithms was evaluated: PSO, GWO and LOA. While each produced interesting results—notably GWO for stability, PSO was the most effective overall. PSO demonstrated an impressive capacity to swiftly locate solutions near the optimum while adapting to the dynamic constraints of multimodal transport. Thanks to its collective intelligence logic inspired by swarm behavior, PSO was able to effectively balance the exploration and exploitation of the search space, which is essential to avoid making irrelevant recommendations. PSO also stood out for its robustness despite increasingly complex mobility data and its simplicity of implementation. These factors fully justify its adoption as the primary method in this multimodal recommendation project.

3.3. Calculating Trajectory Attributes for the Recommendation Phase

In this phase, the calculation essentially involves: for each possible trajectory, the numerical values of the criteria considered important during the recommendation phase (see Figure 9).

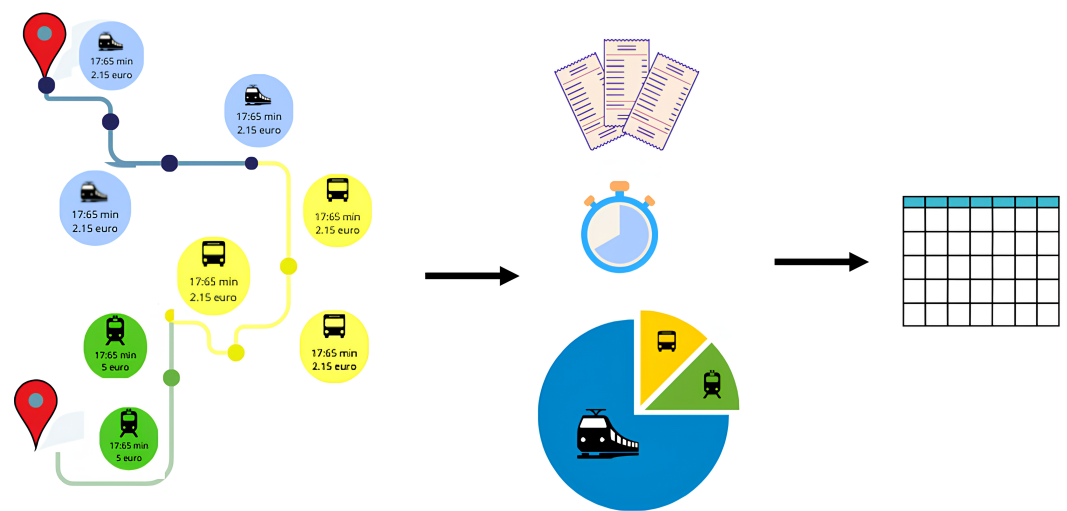


Figure 9. Trajectories attribute computation.

To this end, it is first necessary to compute the proportion of time that each line takes from the entire trajectory; for this, the total duration of the trajectory is (see Formula (3)):

$$\text{Total time of the trajectory} = \sum_{k=0}^n T_k \quad (3)$$

where

- T_k is the time spent on line k ;

- n is the total number of lines in a trajectory.

The proportion of time for each line is then computed (see Formula (4)).

$$\text{Time spent in each type of transportation} = \sum_{k=0}^n \text{Type}_k \cdot t_k \quad (4)$$

where

- t_k is the proportion of time spent on edge k ;
- Type_k is a binary variable that takes the value 1 if that type of transportation mode is used on that line in a trajectory (see Formula (5)).

$$\text{Type}_k = \frac{\text{Time_in_type}}{\text{Tota_time}} \quad (5)$$

where

- Time_in_type is the time spent in each type of transportation;
- Tota_time is the total time of the trajectory.

The cost of the entire trajectory is also required (see Formula (6)).

$$\text{Cost of the whole trajectory} = \sum_{k=0}^n C_k \quad (6)$$

where

- C_k is the cost on line k ;
- n is the total number of lines in a trajectory.

Finally, a data frame is obtained in which each row contains the above information about a generated trajectory.

3.4. Trajectory Recommendation Using RankNet

For the recommendation, an ML solution using a neural network called RankNet was implemented to rank the trajectories. The model takes as input the list of trajectories generated for a specific origin–destination pair, each represented by the attributes counted in the previous step. As a result, the model returns the same list of trajectories but ordered according to the trained model. To evaluate the model, the initial dataset was divided into training and testing sets, with 80% allocated for training and 20% for testing.

RankNet is a pairwise learning-to-rank algorithm developed by Microsoft Research and is primarily used in information retrieval and search engines. It uses a neural network to predict the rank order of items by comparing pairs of items and learning their relative preferences. The network is trained using a gradient descent method to minimize the loss function, which measures the difference between the predicted and actual rankings. During training, RankNet adjusts the weights of the neural network to improve the accuracy of its predictions. This method allows RankNet to effectively handle large-scale ranking problems and produce highly relevant search results. The process shown in Figure 10 begins by creating pairs for each origin–destination pair. For a pair, multiple trajectories are generated, and among these, only one is selected. For each non-selected trajectory, a pair is created in which the selected trajectory is the first element and the non-selected trajectory is the second element. These pairs are then integrated into DataLoader objects to enable efficient batch processing. A RankNet model comprising three fully connected layers was developed to predict the scores for the input feature vectors representing each trajectory. To evaluate the model's performance and facilitate training, a custom loss function specifically designed for RankNet is used.

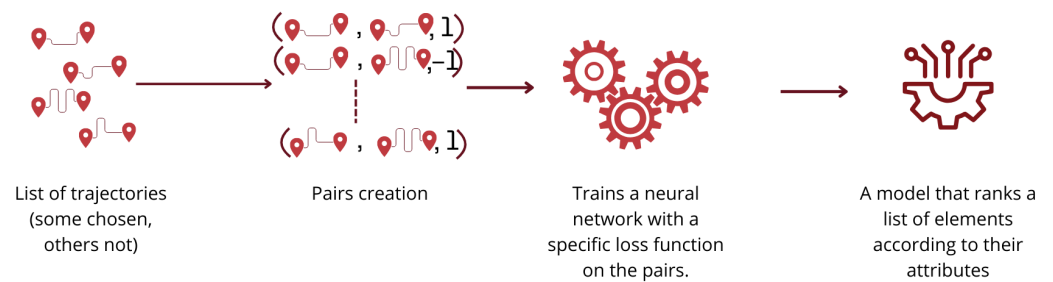


Figure 10. The principles of Ranknet.

The entire process of sorting trajectories is presented in Algorithm 4.

Algorithm 4: Training and Evaluating a RankNet Model

Input: Initial trajectories dataset, Origin–Destination pairs, Number of epochs

Output: Trained RankNet model

Begin

Step 1: Split the data into training and testing sets;

`train_data, test_data = split_data(initial_dataset);`

Step 2: Create pairs of trajectories for each origin–destination pair;

`pairs = [];`

for each od_pair in origin_destinations do

`trajectories = retrieve_trajectories(od_pair, the dataset);`

`selected_trajectory = select_trajectory(trajectories);`

for each traj in trajectories do

if traj ≠ selected_trajectory then

`pairs.append((selected_trajectory, traj));`

end

end

end

Step 3: Create DataLoader for efficient batch processing;

`data_loader = DataLoader(pairs);`

Step 4: Define the RankNet model with three fully connected layers;

`model = RankNetModel(layers=3);`

Step 5: Train the model using a custom RankNet loss function;

for each epoch in range(num_epochs) do

for each batch in data_loader do

`selected_traj, non_selected_traj = batch;`

`score1 = model(selected_traj);`

`score2 = model(non_selected_traj);`

`loss = ranknet_loss(score1, score2);`

`optimize_model(loss);`

end

end

Step 6: Evaluate the model performance on the test data;

`evaluate_model(model, test_data);`

End.

The entire recommendation process is represented in Algorithm 5 which outlines all the steps from capturing the origin and destination to displaying the recommendations.

Algorithm 5: Recommending the best trajectories**Input:** Origin–Destination (O-D) coordinates**Output:** Ordered list of recommended trajectories with the user’s favorite predicted one**Begin**

Add O and D to our network as the departure and destination nodes;

Add links between them and the nearest stations using the Neighbor Station selection Algorithm;

Select the 8 optimal trajectories using the Practice Swarm Optimization Algorithm;

Update and finalize the trajectories and return the dictionary of lines of each one using the finalizing trajectories algorithm;

Use the returned dictionary to create a dataframe of attributes with a trajectory in each line;

Use the RankNet model to recommend one of the generated trajectories

represented in the generated dataframe to the user and display the other options;

Take the final choice of the user and update their profile for further recommendations;

End.

The suggested approach integrates PSO and RankNet for trajectory recommendation in multimodal transportation systems. By considering factors such as travel time, cost, and user preferences, our approach aims to enhance the effectiveness and accuracy of trajectory RSs, ultimately improving the overall user experience in navigating transportation networks.

4. Experiment and Evaluation

In this section, the implementation and evaluation of the system are presented. The datasets used in both the trajectory generation and recommendation phases are first detailed. Next, the development environment employed for system realization is described. Finally, the results of the evaluation conducted for all steps and phases are presented.

To protect user data privacy during the collection and processing of origin–destination pairs and trajectory selections, measures were implemented. First, anonymity techniques are applied to remove or mask personally identifiable information, ensuring that individual users cannot be directly identified from the datasets. Aggregation methods are also used to analyze data at a group level rather than an individual level, thereby reducing privacy risks. Additionally, secure data transmission protocols, such as encryption during data transfer and storage, safeguard the information against unauthorized access. Access controls and strict data governance policies limit who can view or handle sensitive data.

4.1. Dataset Description

For the dataset of the transportation network, data sourced from the “Open Data portal of Île-de-France Mobilités” was utilized, which provides comprehensive open data concerning mobility in the Île-de-France area [30]. Specifically, the dataset comprises 72,804 rows, detailing all lines within the Île-de-France network and the corresponding stops served by each line. Initially, this dataset contained the following attributes:

- route id: The identifier of the line.
- route long name: The name of the line.
- stop id: The identifier of a station.
- stop name: The name of the station.
- stop lon: The longitude of the station.

- stop lat: The latitude of the station.
- OperatorName: The name of the operator.
- Nom commune: The name of the municipality.
- Code insee: The municipality code.
- Pointgeo: The geographic point of the station.

Figure 11 illustrates the initial state of the dataset.

	route_id	route_long_name	stop_id	stop_name	stop_lon	stop_lat	OperatorName	Pointgeo	Nom_commune	Code_insee
0	IDFM:C00001	482	IDFM:2679	Lycée Marianne	2.427407	48.730014	Keolis Ouest Val-de-Marne	48.730013592071785, 2.4274068162228435	Villeneuve-le-Roi	94077
1	IDFM:C00001	482	IDFM:2709	EDF	2.438983	48.729511	Keolis Ouest Val-de-Marne	48.729510906921725, 2.4389825601620467	Villeneuve-le-Roi	94077
2	IDFM:C00001	482	IDFM:461967	Ecoles	2.440496	48.732918	Keolis Ouest Val-de-Marne	48.73291790476698, 2.440496016072866	Villeneuve-le-Roi	94077
3	IDFM:C00001	482	IDFM:463501	Marché	2.420823	48.736440	Keolis Ouest Val-de-Marne	48.736439826111166, 2.420822990662051	Villeneuve-le-Roi	94077
4	IDFM:C00001	482	IDFM:21660	Place Amédée Soupault	2.418142	48.734232	Keolis Ouest Val-de-Marne	48.734232149423555, 2.4181416099829285	Villeneuve-le-Roi	94077
...	...	...	...	...	...	...	...	...	...	...
72799	IDFM:C01841	P	IDFM:472235	Changis - Saint-Jean	3.024129	48.958124	SNCF	48.958123608993006, 3.024129201681532	Changis-sur- Marne	77084
72800	IDFM:C01841	P	IDFM:472311	Mortcerf	2.908207	48.788625	SNCF	48.78862538612734, 2.908206738274958	Mortcerf	77318
72801	IDFM:C01841	P	IDFM:483833	Sainte- Colombe Septveilles / Place du Général ...	3.254041	48.529332	SNCF	48.52933156603378, 3.2540414434793448	Sainte-Colombe	77404
72802	IDFM:C01841	P	IDFM:470684	Nangis	3.012481	48.561285	SNCF	48.56128481096427, 3.012481441812586	Nangis	77327
72803	IDFM:C01841	P	IDFM:6756	Gare de Verneuil l'Étang	2.825089	48.644848	SNCF	48.644848150315916, 2.8250892153714657	Verneuil-l'Étang	77493

72804 rows × 10 columns

Figure 11. Initial state of the dataset.

Initially, the dataset included stations and transport lines from several cities in France. However, for this study, the focus is on a single city, which is sufficient for the evaluation. Moreover, the city itself has more than 6000 stations, making it preferable to limit the number of stations from other cities to reduce complexity. Since it is preferable to select a city with a large number of stations to enable the generation of multiple trajectory options, the number of stations available for each line is displayed in Figure 12 and the stations on the France map are referenced to observe their distribution [31]. The results are illustrated in Figure 13.

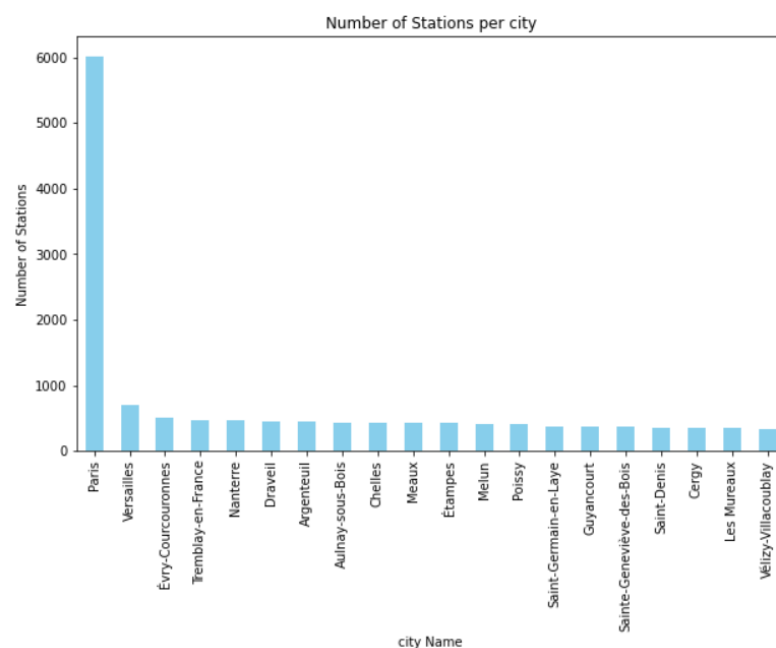


Figure 12. Distribution of stations in cities.

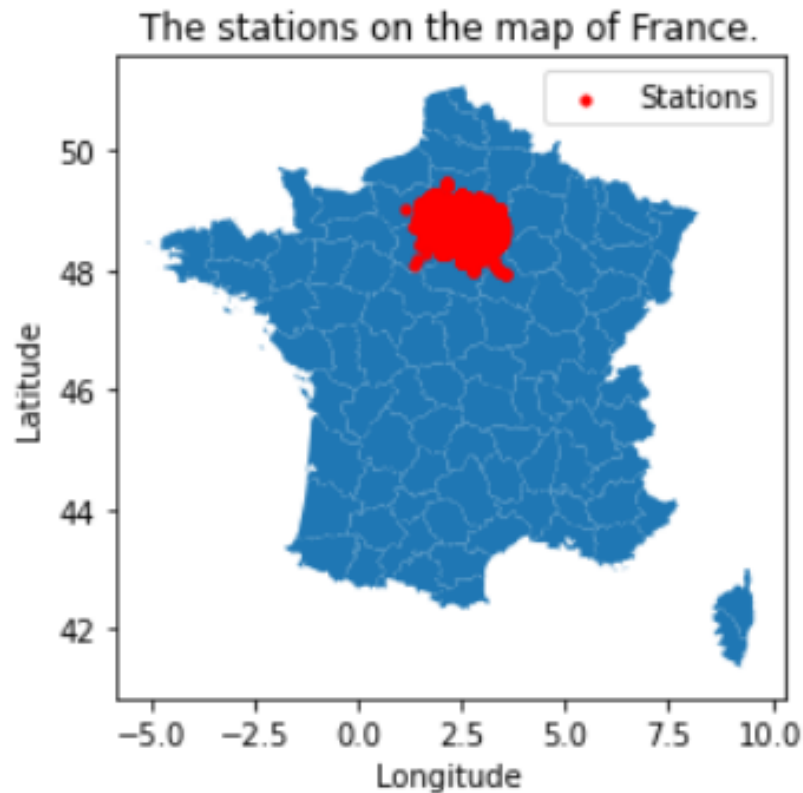


Figure 13. Location of stations on the map of France.

Observing the results, it can be seen that most of the stations are located in the city of Paris. Therefore, this city is selected including only its lines and stations, reducing and optimizing therefore the dataset from 72,804 to 6019 stations. Once the city was identified, we moved on to the data cleaning phase by removing unnecessary data and adding missing information.

For this study, the mode of transportation for each line is required, which is not available in the current dataset. To address this issue, for each line, the name of one of its stations is retrieved and the type of public transportation for the station is manually verified. This type of transportation is then assigned to the entire line.

Since the goal is to model the transport network as a graph and the dataset of stations representing the nodes is already available, the links between these nodes, which constitute the edges, are required. To this end, an Edges dataset is incorporated, defining an edge between each pair of consecutive stations on the same line.

For trajectory recommendation, since our RS is content-based, focusing on the analysis of item attributes, it is necessary to have trajectories previously selected by the user from a group of trajectories to utilize it. Trajectories were generated for 100 origin–destination pairs using the trajectory generation algorithm. The algorithm returned between 1 and 8 trajectories for each pair. One trajectory per pair was then manually selected to reflect the user’s preference using a form-based selection. As a result, a dataset comprising 514 trajectories was obtained, of which 100 were selected (one per pair) as user choices, and the remaining were not chosen. Each entry involves the attributes returned by the trajectory generation algorithm, which are as follows:

- Origin: Origin coordinates.
- Destination: Destination coordinates.
- Duration (Hour): Travel duration.
- Cost: Total cost.
- Distance: Distance of the journey.
- Proportion “mode”: Proportion of each mode of transport from 0 to 1.
- chosen: Boolean variable indicating whether the trajectory was taken or not.

The Streamlit library (Version 1.50.0) in Python is used to present the trajectory RS [32]. The proposed interface primarily features a map of Paris, which is shown in Figure 14, which is directly displayed to the user, prompting them to select their starting point and then their destination on the map.

Route Recommendation System

Please click on your starting point and then your destination point on the map. Then click the 'Generate Routes' button.

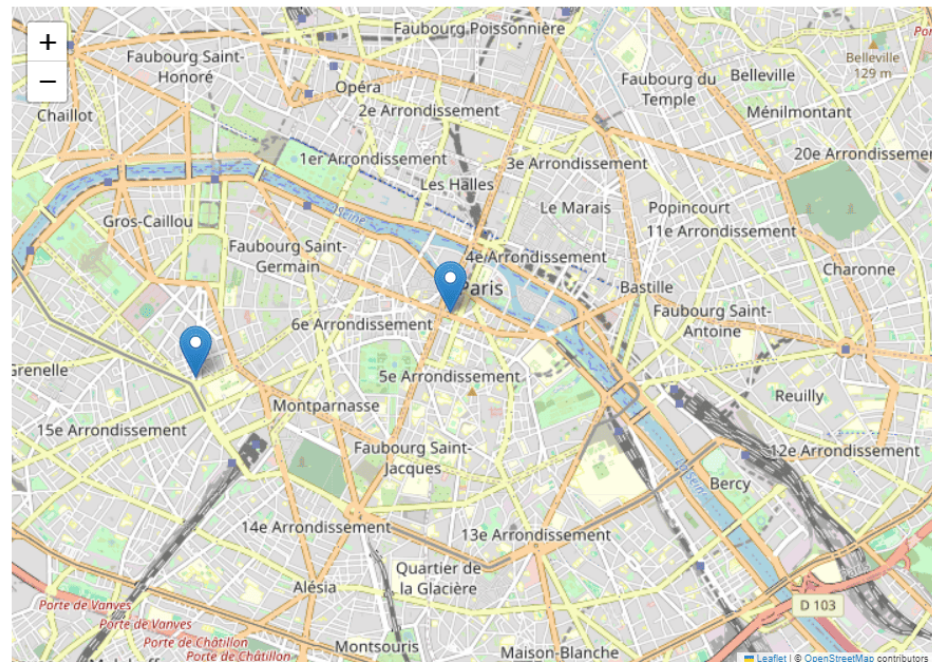


Figure 14. Selecting starting and destination points on the map.

After the user selects the coordinates, the system retrieves these coordinates and displays them back to the user, as illustrated in the following example:

Starting Point: 'lat': 48.85127481320342, 'lng': 2.3443794250488286

Destination Point: 'lat': 48.84573966784039, 'lng': 2.3112487792968754

After the coordinates are selected and displayed, the user simply needs to press the button to generate trajectories. The system will then produce a list of ordered generated trajectories, according to the recommendation model. The trajectories will be displayed with a gradient of green shades, as highlighted in Figure 15, where lighter shades of green indicate a higher recommendation level.

Route Recommendation System

The trajectory will take 18 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 3 stations from IDFM:23662 to IDFM:23658 for 1 minutes and 2.15 €.

Walk : to destination for 16 minutes.

Select 0

The trajectory will take 16 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 5 stations from IDFM:23662 to IDFM:23655 for 1 minutes and 2.15 €.

Walk : to destination for 14 minutes.

Select 1

The trajectory will take 15 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 6 stations from IDFM:23662 to IDFM:23653 for 1 minutes and 2.15 €.

Walk : to destination for 13 minutes.

Select 1

The trajectory will take 15 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 6 stations from IDFM:23662 to IDFM:23653 for 1 minutes and 2.15 €.

Walk : to destination for 13 minutes.

Select 2

The trajectory will take 8 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 15 stations from IDFM:23662 to IDFM:36665 for 5 minutes and 2.15 €.

Walk : to destination for 2 minutes.

Select 3

The trajectory will take 7 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 14 stations from IDFM:23662 to IDFM:426942 for 4 minutes and 2.15 €.

Walk : to destination for 1 minutes.

Select 4

The trajectory will take 7 minutes and will cost 2.15€

Walk : to IDFM:23662 for 1 minutes.

Take the metro line 56 : for 13 stations from IDFM:23662 to IDFM:426940 for 4 minutes and 2.15 €.

Walk : to destination for 2 minutes.

Select 5

The trajectory will take 22 minutes and will cost 4.3€

Walk : to IDFM:21910 for 1 minutes.

Take the metro line 9 : for 2 stations from IDFM:21910 to République for 2 minutes and 2.15 €.

Take the metro line 56 : for 5 stations from République to IDFM:23655 for 5 minutes and 2.15 €.

Walk : to destination for 14 minutes.

Select 6

Go Back

Figure 15. Generated trajectories in a predefined sequence.

After generating and displaying the trajectories, the user will need to press the ‘Select i’ button next to the selected trajectory i so that this trajectory is added to his selected trajectories history and to obtain the details of this trajectory printed on a map as shown in Figure 16. To identify the type of transportation for each trajectory, each distinct type is

assigned a display color, and clarification of the colors for each type is provided as shown in the legend of Figure 16 to the user alongside the display of their trajectories.

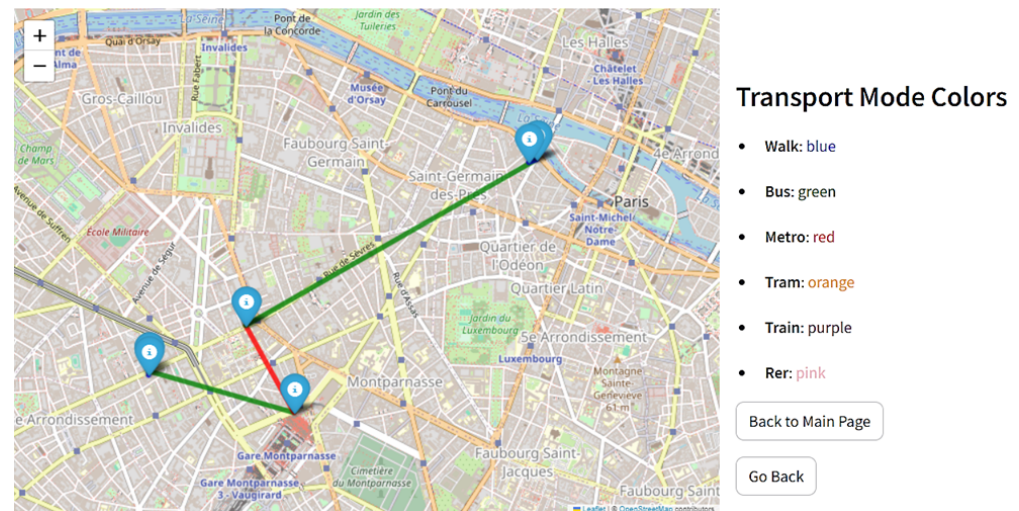


Figure 16. Displaying the details of the trajectory. The figure shows the original screenshot including color scheme. Note that some transport modes are not used in the shown solution.

4.2. Evaluation

In this section, the results of the evaluation of the suggested approach are presented. The system consists of two main components: trajectory generation and recommendation of the best trajectory according to user preferences. The approach was evaluated in two ways: first, in terms of the relevance of the generated trajectories, and second, in terms of the recommendations.

Evaluation of Trajectory Generation

To assess the quality of the generated trajectories, the metrics considered essential were used as follows: For the first two metrics, a dataset consisting of trajectories generated for 100 origin–destination pairs was used. The coverage of all transportation modes in the dataset was evaluated by displaying the frequency of each transportation mode, followed by the frequency of the combinations provided by the system. The results are presented in Figures 17 and 18.

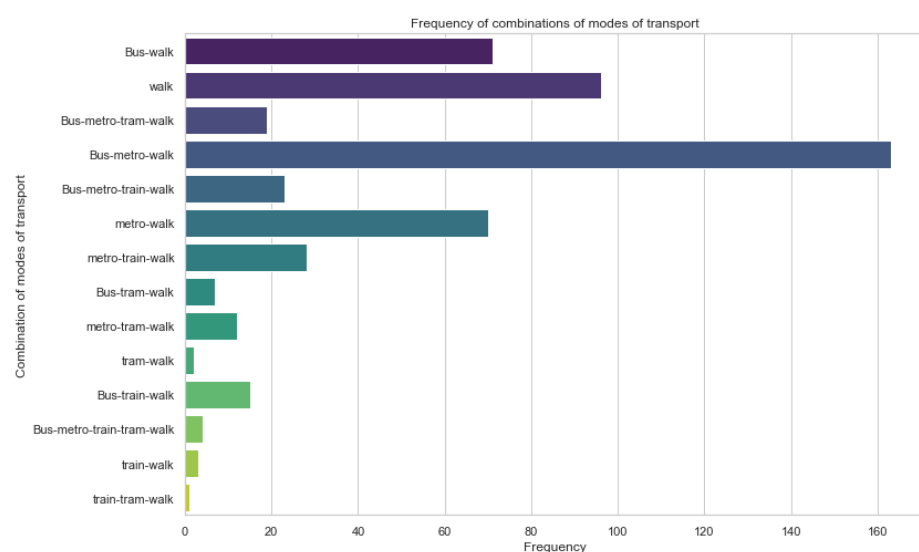


Figure 17. Frequency of combinations of modes of transport.

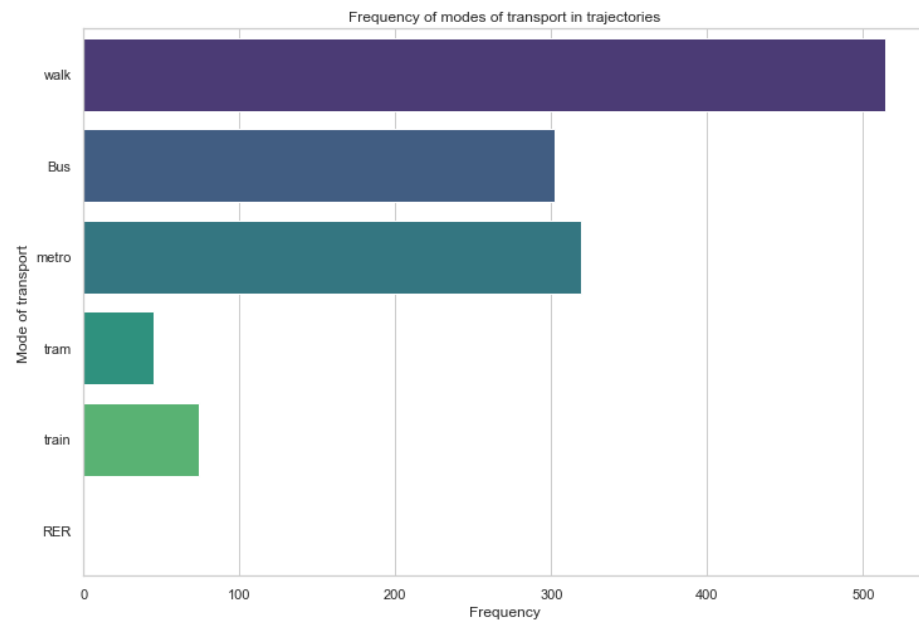


Figure 18. Frequency of modes of transport in trajectories.

Regarding the results, it can be observed that the system covers all modes of transport except for the RER. This is likely due to the small number of lines in the dataset (less than 5 lines). The most common mode of transportation is the metro, followed by the bus, reflecting the recent trend in Paris where the metro is the most frequently used mode of transport. Regarding transportation combinations, the most frequent combination observed is Bus-Metro, with a total of 9 combinations involving more than one mode of transport, which is quite significant.

The execution time for generating trajectories for 20 origin–destination pairs, from request initiation to the results, was then recorded. the results are illustrated in Figure 19.

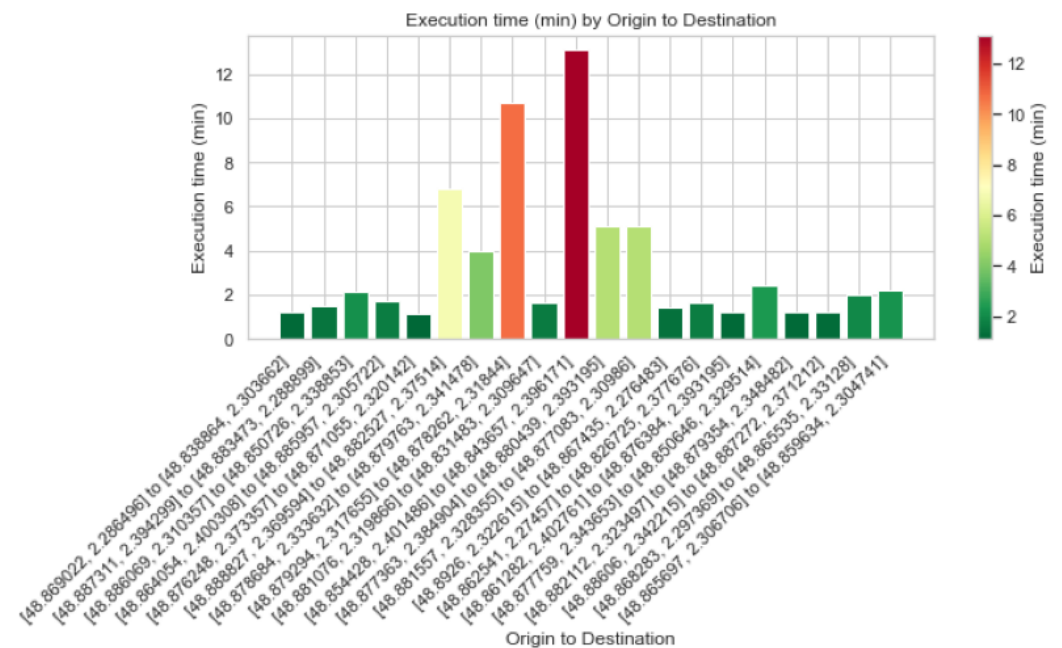


Figure 19. Execution time for 20 origin–destination pairs.

The execution time varied between 1 and 10 min, depending on the selected origin–destination pair. In addition, the application was run locally on a laptop with average specifications. Consequently, the generation speed is influenced by the complexity of the

chosen route. Overall, the algorithm demonstrates a satisfactory performance in terms of the trajectory generation time. The average distance of the generated trajectories for each pair, as well as the minimum distance among these trajectories, was then displayed, based on the straight-line distance between the origin and destination points: Figure 20 displays the outcomes.

Finally, the trajectories generated by the algorithm for these 20 pairs were compared with those generated by Google Maps, aiming to evaluate both travel time and cost. The results from Google Maps were obtained by manually searching for the trajectories for each of the 20 origin–destination pairs and recording the minimum travel time and cost. It is important to note that the comparison is not entirely accurate in terms of time, as our system does not account for waiting times and transport schedules. Additionally, it should be noted that the execution resources are not identical, as the tests were conducted on the local machine infrastructure. Results are depicted in Figure 21.

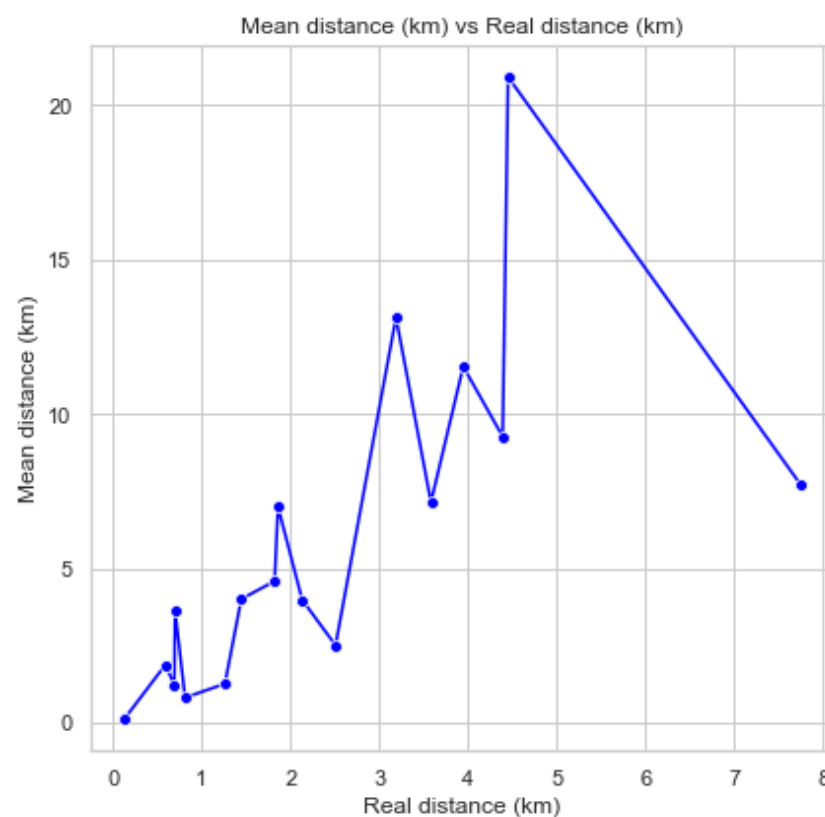


Figure 20. Average trajectory distances compared to the direct point-to-point distances.

The following metrics are used for the ranking recommendation [33,34] which are usually used for RSs. These metrics capture a different aspect of performance essential for user satisfaction and system effectiveness.

1. **Weighted Precision:** Measures the proportion of true positives among the items predicted as positives, considering class weights (see Formula (7)). Useful for imbalanced classes.

$$\text{Weighted Precision} = \frac{\sum_{i=1}^k \text{Precision}_i \cdot n_i}{\sum_{i=1}^k n_i} \quad (7)$$

2. Recall: Measures the proportion of true positives among all actual positives (see Formula (8)). Indicates the model's ability to detect true positives.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (8)$$

3. Weighted F1 Score: The harmonic mean of the weighted precision and recall, considering class weights (see Formula (9)). Balances precision and recall for imbalanced data.

$$\text{Weighted F1 Score} = \frac{2 \cdot \text{Weighted Precision} \cdot \text{Weighted Recall}}{\text{Weighted Precision} + \text{Weighted Recall}} \quad (9)$$

4. Mean Reciprocal Rank (MRR): Measures search/recommendation effectiveness, averaged over the reciprocal ranks of the first relevant results (see Formula (10)). This metric is useful for ranking tasks.

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (10)$$

5. AUC-ROC (Area Under the Receiver Operating Characteristic Curve): Evaluates binary classification performance by measuring the area under the ROC curve, plotting the true positive rate against the false positive rate. The AUC is typically calculated using numerical integration methods.

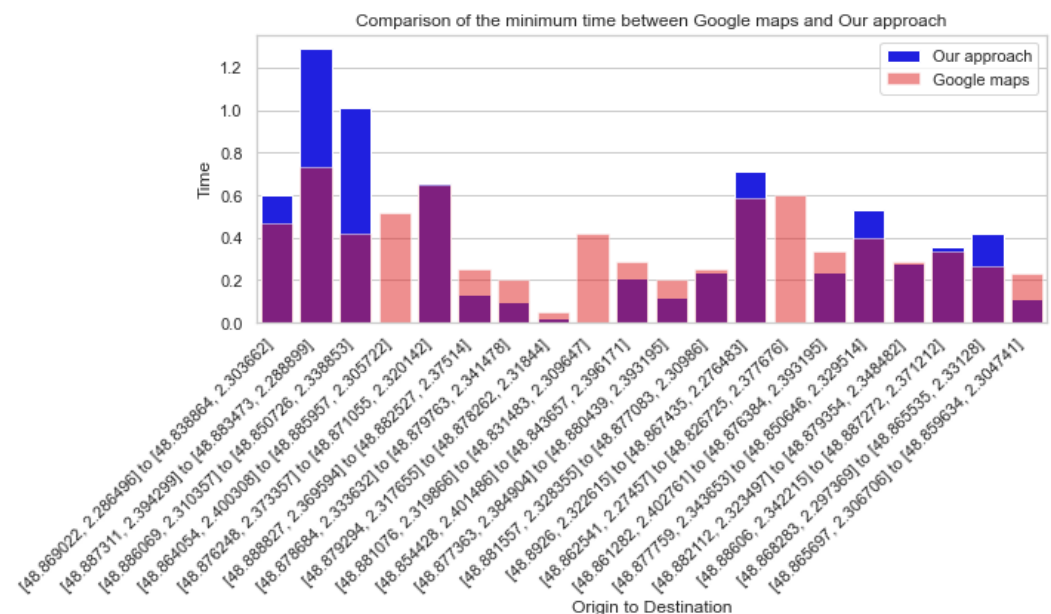


Figure 21. Comparison of the minimum time between Google Maps and the proposed approach.

The results of the experiment are presented in Table 1.

Table 1. Performance metrics for the ranking recommendation.

Metric	Value
Weighted Precision	0.9820
Recall	0.9400
Weighted F1 Score	0.9600
Mean Reciprocal Rank (MRR)	1.0000
AUC-ROC	0.9100

Table 1 presents the performance metrics of the ranking-based RS. The model demonstrates good performance across several key evaluation metrics, marking a significant advancement in the area of smart city transport prediction. This improvement is particularly important because accurate and timely transport forecasts are central to optimizing urban mobility, reducing congestion, and improving commuter experiences. The Weighted Precision remains high at 0.9820, indicating that the majority of the model's positive predictions—such as identifying periods of congestion or system delays—are correct. Although this value is slightly lower than before, it still reflects excellent precision and is consistent with the increase in recall. Recall has significantly improved to 0.9400, which means the model is now more effective at detecting a broader range of relevant transport events. This improvement is crucial in real-world applications, where missing a critical delay or disruption could negatively impact traffic flow or public transit reliability. The Weighted F1 Score has also risen to 0.9600, demonstrating a stronger overall balance between precision and recall. This suggests that the model is not only accurate in its predictions but also consistent in capturing most of the important transport-related instances. Additionally, the AUC-ROC score obtained is 0.9100, showing that the model's ability to distinguish between different operational states—such as normal flow versus congestion—has become more refined. A higher AUC-ROC value indicates a more capable classifier, especially in complex, real-time environments. Taken together, these results indicate that the updated model offers a more reliable and robust predictive framework for transport in smart cities. This represents a meaningful step forward in the development of intelligent mobility systems, supporting better decision-making, resource allocation, and urban planning. These results are considered to be high-performing compared to the state of the art, which demonstrates the relevance of our approach.

In summary, the model demonstrates excellent precision and ranking quality, with slightly lower but still strong recall and discriminative capability.

5. Conclusions

Multimodal transport recommendation entails proposing a route for users from a starting point to a destination, comprising one or more available transport modes, while preserving user preferences. The context was first provided, and the background of recommendation systems was discussed, their principles, methods used, and evaluation metrics. The focus then shifted to smart cities, as their development has greatly facilitated the emergence of such systems. In the following section, the concept of multimodal transport systems and the transformation from conventional to multimodal transport was explored.

The most recent research in the field of recommendation systems was analyzed to comprehend the latest developments and advancements, recognizing the diversity of approaches. The proposed approach, consisting of two main steps, is then presented. Initially, the network was characterized as a graph, where each station serves as a node. An optimal method for trajectory generation was then sought, with PSO being chosen. Once the trajectories were generated, post-processing was applied to ensure their feasibility. Finally, these trajectories were recommended to users based on previously selected trajectories by training a ranking model.

The model was implemented and tested using real-world data from the Paris transport network, which required some preprocessing to be usable. Trajectories were generated for origin–destination pairs, manual selections were made among the proposals, and the recommendation model was evaluated on this dataset. The model's results were generally good, producing feasible trajectories with reasonable prices and travel times. However, execution times varied, sometimes taking longer for certain trajectories and less for others. The returned trajectories were also compared with those from Google Maps in terms of

cost and time. Although the recommendation evaluation was based on a small dataset, the results were very promising, with nearly perfect accuracy.

The proposed multimodal transport RS encounters substantial computational limitations within extensive urban networks, primarily due to the complexity of optimizing routes across various modes, fluctuating traffic conditions, and user preferences. Real-time performance is compromised by the requirement to swiftly process substantial volumes of data (e.g., transit schedules, GPS and traffic feeds) to compute optimal paths. Algorithms such as PSO offer better scalability and faster convergence, making them more suitable for near real-time applications. Efficient data handling, parallel computing, and algorithmic optimizations are essential for maintaining responsiveness and accuracy on a large scale.

To further improve the proposed model, hybrid optimization methods will be explored in future studies to reduce trajectory search times, and transport schedules will be incorporated for better adaptability. Additionally, integrating transport subscription options would provide more realistic and contextually relevant recommendations within the transport network. In addition, several key areas have been identified for future research, which is aimed at improving the system, with a focus on adaptability, scalability, and responsiveness. Real-time traffic and transportation data, including live congestion levels, public transportation delays, and service disruptions, is a key area of focus. Integrating this dynamic data would enable the system to provide more accurate and context-appropriate route recommendations, particularly in constantly changing urban environments. Another important area is geographic expansion. Extending the system to other cities or regions requires flexible data pipelines and scalable algorithms that can adapt to different transportation infrastructures, user behaviors, and data formats. This would enable wider deployment and increase the system's impact. Future research could also explore personalized routing using machine learning and user behavior data to tailor suggestions to individual travel habits and preferences. Improving the interoperability with other smart city systems and Mobility as a Service (MaaS) platforms is another promising avenue.

Author Contributions: Conceptualization, H.E.B. and M.M.; methodology, H.E.B. and M.M.; software, M.M.; validation, H.E.B. and M.M.; formal analysis, H.E.B. and M.M.; investigation, M.M.; resources, H.E.B., M.M. and T.H.; writing—original draft preparation, M.M., H.E.B. and T.H.; writing—review and editing, T.H.; visualization, M.M.; supervision, H.E.B.; project administration, H.E.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed at the corresponding author

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. El Bouhissi, H. Recommendation Systems. In *Encyclopedia of Data Science and Machine Learning*; IGI Global: Hershey, PA, USA, 2022; pp. 2839–2855. <https://doi.org/10.4018/978-1-7998-9220-5.ch169>.
2. Andrade-Ruiz, G.; Carrasco, R.A.; Porcel, C.; Serrano-Guerrero, J.; Mata, F.; Arias-Oliva, M. Emerging Perspectives on the Application of Recommender Systems in Smart Cities. *Electronics* **2024**, *13*, 1249. <https://doi.org/10.3390/electronics13071249>.
3. El Bouhissi, H.; Tagzirt, D.; Bouredjioua, F.; Pavlova, O. Health Recommender System for Smart Cities. In Proceedings of the MoMLet+ DS, Lviv-Leiden, Ukraine, 3–4 June 2023; pp. 334–343.
4. Ivaldi, E.; Pavanini, T.; Filì, T.; Musso, E. National Models of Smart City Development: A Multivariate Perspective on Urban Innovation and Sustainability. *Sustainability* **2025**, *17*, 7420. <https://doi.org/10.3390/su17167420>.

5. Khanmohamadi, M.; Guerrieri, M. Smart Intersections and Connected Autonomous Vehicles for Sustainable Smart Cities: A Brief Review. *Sustainability* **2025**, *17*, 3254. <https://doi.org/10.3390/su17073254>.
6. Liang, Y.; You, J.; Wang, R.; Qin, B.; Han, S. Urban Transportation Data Research Overview: A Bibliometric Analysis Based on CiteSpace. *Sustainability* **2024**, *16*, 9615. <https://doi.org/10.3390/su16229615>.
7. Billones, R.K.C.; Guillermo, M.A.; Lucas, K.C.; Era, M.D.; Dadios, E.P.; Fillone, A.M. Smart Region Mobility Framework. *Sustainability* **2021**, *13*, 6366. <https://doi.org/10.3390/su13116366>.
8. Hasan, M.; Fadli, F. Multimodal Transport For Smart Mobility in Emerging Cities: Case of Doha. In Proceedings of the International Conference on Civil Infrastructure and Construction (CIC), Doha, Qatar, 5–8 February 2023; pp. 933–940. <https://doi.org/10.29117/cic.2023.0120>.
9. Rocha, N.P.; Dias, A.; Santinha, G.; Rodrigues, M.; Queirós, A.; Rodrigues, C. Smart mobility: A systematic literature review of mobility assistants to support multi-modal transportation situations in smart cities. In Proceedings of the International Conference on Integrated Science, Kep, Cambodia, 1–3 May 2020; Springer: Cham, Switzerland, 2021; pp. 303–312.
10. Kramarz, M.; Przybylska, E. Multimodal transport in the context of sustainable development of a city. *Sustainability* **2021**, *13*, 2239.
11. Krúpová, S.; Koman, G.; Soviar, J.; Holubčík, M. The Role of Business Models in Smart-City Waste Management: A Framework for Sustainable Decision-Making. *Systems* **2025**, *13*, 556. <https://doi.org/10.3390/systems13070556>.
12. Saifudin, I.; Widiyaningtyas, T. Systematic Literature Review on Recommender Systems: Approach, Problems, Evaluation Techniques, and Datasets. *IEEE Access* **2024**, *12*, 19827–19847. <https://doi.org/10.1109/ACCESS.2024.3359274>.
13. Ko, H.; Lee, S.; Park, Y.; Choi, A. A Survey of Recommendation Systems: Recommendation Models, Techniques, and Application Fields. *Electronics* **2022**, *11*, 141. <https://doi.org/10.3390/electronics11010141>.
14. El Bouhissi, H.; Patel, A.; Debnath, N.C. Recommender System for E-Commerce: How Ontologies Support Recommendations. In *Data Science with Semantic Technologies*; CRC Press: Boca Raton, FL, USA, 2023; pp. 287–297.
15. Saini, K.; Singh, A. A content-based recommender system using stacked LSTM and an attention-based autoencoder. *Meas. Sens.* **2023**, *31*, 100975. <https://doi.org/10.1016/j.measen.2023.100975>.
16. Rajput, S.; Mehta, N.; Singh, A.; Keshavan, R.; Vu, T.; Heldt, L.; Hong, L.; Tay, Y.; Tran, V.Q.; Samost, J.; et al. Recommender Systems with Generative Retrieval. In Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023), New Orleans, LA, USA, 10–16 December 2023.
17. Liu, Y.; Lyu, C.; Liu, Z.; Cao, J. Exploring a large-scale multi-modal transportation recommendation system. *Transp. Res. Part C Emerg. Technol.* **2021**, *126*, 103070.
18. Xu, A.; Zhong, P.; Kang, Y.; Duan, J.; Wang, A.; Lu, M.; Shi, C. THAN: Multimodal Transportation Recommendation with Heterogeneous Graph Attention Networks. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 1533. <https://doi.org/10.1109/TITS.2022.3221370>.
19. Campigotto, P.; Rudloff, C.; Leodolter, M.; Bauer, D. Personalized and Situation-Aware Multimodal Route Recommendations: The FAVOUR Algorithm. *IEEE Trans. Intell. Transp. Syst.* **2016**, *18*, 92–102. <https://doi.org/10.1109/TITS.2016.2565643>.
20. Hao, C.; Yue, Y. Optimization on Combination of Transport Routes and Modes on Dynamic Programming for a Container Multimodal Transport System. *Procedia Eng.* **2016**, *137*, 382–390. <https://doi.org/10.1016/j.proeng.2016.01.272>.
21. Liu, H.; Li, T.; Hu, R.; Fu, Y.; Gu, J.; Xiong, H. Joint Representation Learning for Multi-Modal Transportation Recommendation. In Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-19), Honolulu, HI, USA, 27 January–1 February 2019; AAAI Press: Washington, DC, USA, 2019.
22. Liu, H.; Tong, Y.; Han, J.; Zhang, P.; Lu, X.; Xiong, H. Incorporating Multi-Source Urban Data for Personalized and Context-Aware Multi-Modal Transportation Recommendation. *IEEE Trans. Knowl. Data Eng.* **2020**, *32*, 2343–2355.
23. Liu, H.; Han, J.; Fu, Y.; Li, Y.; Chen, K.; Xiong, H. Unified route representation learning for multi-modal transportation recommendation with spatiotemporal pre-training. *VLDB J.* **2023**, *32*, 325–342. <https://doi.org/10.1007/s00778-022-00748-y>.
24. Yang, R.; Li, D.; Han, B.; Zhou, W.; Yu, Y.; Li, Y.; Zhao, P. Door to door space-time path planning of intercity multimodal transport network using improved ripple-spreading algorithm. *Comput. Ind. Eng.* **2024**, *189*, 109996. <https://doi.org/10.1016/j.cie.2024.109996>.
25. Ghalmane, Z.; Daoud, B. Graph-Based Learning for Multimodal Route Recommendation. In Proceedings of the International Conference on Agents and Artificial Intelligence (ICAART), Porto, Portugal, 23–25 February 2025; SciTePress: Setúbal, Portugal, 2025; pp. 133–142.
26. Yu, Y.; Li, D.; Han, B.; Zhang, Q.; Huang, Y.; Yang, R. Enhancing urban mobility: A multi-modal travel plan recommendation framework integrating the influences of temporal characteristics and candidate sets. *Inf. Sci.* **2025**, *709*, 122042. <https://doi.org/10.1016/j.ins.2025.122042>.
27. Yu, J.; Huang, J.; Chi, L.; Wei, W.; Cui, Z. A Traffic Trajectory Recommendation Scheme Based on Edge-Cloud Computing Driver-Car-Road Preferences. *Informatica* **2025**, *36*, 223–240. <https://doi.org/10.15388/25-INFOR585>.
28. Altun, G.; Aydın, İ. Optimizing Unmanned Vehicle Navigation: A Hybrid PSO-GWO Algorithm for Efficient Route Planning. *Firat Univ. J. Exp. Comput. Eng.* **2025**, *4*, 100–114. <https://doi.org/10.62520/fujece.1501508>.

29. Liu, J.; Peng, X.; Huang, M.; Ma, Y.; Jiang, C.; Hu, W.; Zhang, J. Identification and Forecasting of Key Influencing Factors in China's Agricultural Carbon Emissions: Based on Machine Learning Method. *Systems* **2025**, *13*, 554. <https://doi.org/10.3390/systems13070554>.
30. Île-de France Mobilités. Portail Open Data—Données de mobilité en Île-de-France. Available online: <https://www.iledefrance-mobilites.fr/open-data> (accessed on 10 November 2025).
31. OpenStreetMap Contributors. Map of France. 2025. Available online: <https://www.openstreetmap.org/relation/1403916> (accessed on 28 July 2025).
32. Treuille, A.; Teixeira, T.; Kelly, A.; Streamlit, Inc. *Streamlit: The Fastest Way to Build Data Apps*, version 1.x; Streamlit: San Francisco, CA, USA, 2019. Available online: <https://streamlit.io> (accessed on 28 July 2025).
33. Jadon, A.; Patil, A. A Comprehensive Survey of Evaluation Techniques for Recommendation Systems. *arXiv* **2024**, arxiv:2312.16015.
34. Ovono, G.; Zuva, T.; Otunniyi, T. A Review of Evaluation Metrics Algorithmic Used in Scholarly Research Recommender System. In Proceedings of the 2024 4th International Multidisciplinary Information Technology and Engineering Conference (IMITEC), Vanderbijlpark, South Africa, 27–29 November 2024; pp. 416–423. <https://doi.org/10.1109/IMITEC60221.2024.10851124>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.